# Si nos organizamos, nos enamoramos todos

Data Science and decision-making — in dating applications

Gonzalo Barrera Borla

March 18, 2026 — 19:30
MCD10 — Applications of data science in the private sector
Instructor: Pablo Mislej — Universidad de San Andrés

# Who am I?

Gonzalo Barrera Borla

B.A. Economics — FCE, UBA

(almost) M.S. Statistics — FCEyN, UBA

Data Scientist at Sitch

# Sitch

- Dating app with **AI-powered** matchmaking
- No swiping: the AI learns your preferences and suggests candidates with **context**
- Profiles reviewed by real people
- When there's a mutual match, the AI introduces both — and steps aside

download.joinsitch.com

# Data Science “in the wild”

# Prediction is not enough

- In school we learn to **estimate** and **predict**
- In practice, we need to **make decisions** with those predictions
- The objective function is often complex — it's not just "maximize expected value"

# Prediction is not enough

- In school we learn to **estimate** and **predict**
- In practice, we need to **make decisions** with those predictions
- The objective function is often complex — it's not just "maximize expected value"

Three domains where this is clear:

1. **Adtech**: Which ad do I show?
2. **Horse racing**: How much do I bet?
3. **Matchmaking**: Which pairs do I present?

# Adtech: the problem

Simplified: 2 clients competing for the same audience

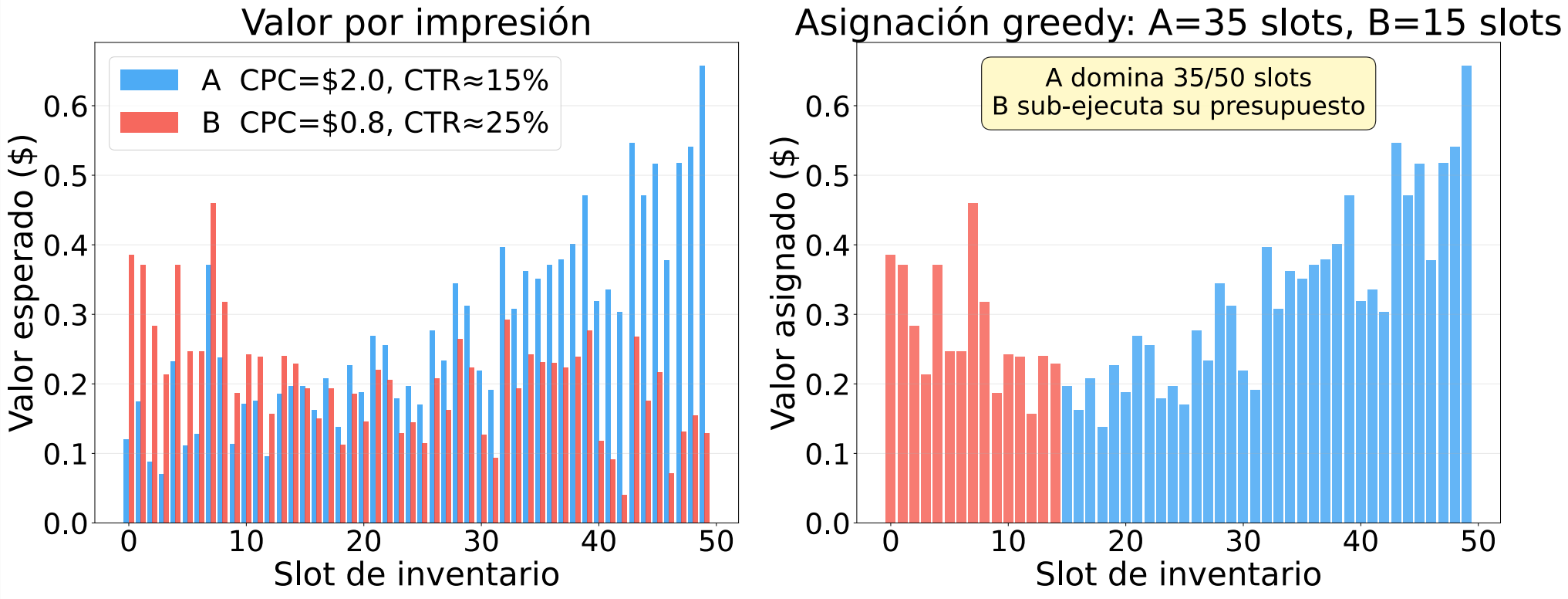
- Client A: high CPC ($2.0), moderate CTR ( 15%)
- Client B: low CPC ($0.8), high CTR ( 25%)
- Both have limited daily budgets

Objective: maximize total revenue

$$\max \sum_i \text{cpc}_i \times \text{clicks}_i \quad \text{s.t.} \quad \sum_t \text{cpc}_i \times x_{i,t} \leq B_i$$

- $x_{i,t} \in \{0, 1\}$: show client $i$'s ad on impression $t$?
- $B_i$: daily budget for client $i$
- $\text{clicks}_i = \sum_t x_{i,t} \times \text{ctr}_{i,t}$: expected clicks

# Adtech: what happens in practice?



# Adtech: what happens in practice?

When $\mathrm{ctr}_A \times \mathrm{cpc}_A > \mathrm{ctr}_B \times \mathrm{cpc}_B$ consistently, A captures nearly all inventory and B under-spends its budget.

# Adtech: what happens in practice?

When $\mathrm{ctr}_A \times \mathrm{cpc}_A > \mathrm{ctr}_B \times \mathrm{cpc}_B$ consistently, A captures nearly all inventory and B under-spends its budget.

The general optimal bidding problem under budget constraints can be solved via Lagrange duality (Pita, 2019).

# Horse racing: Kelly Criterion

The optimal fraction maximizes the expected logarithmic growth of capital (Kelly, 1956):

$$K = \frac{\text{edge}}{\text{odds} - 1} \quad \text{where} \quad \text{edge} = c \times \text{odds} - 1$$

- $K$: fraction of capital to bet ($K < 0 \rightarrow$ don't bet)
- $c$: estimated probability of winning (our model)
- $\text{odds}$: gross payout per \$1 bet (stake included)
- $\text{edge} < 0$: unfavorable game; $= 0$: fair game; $> 0$: bet $K$

# Horse racing: Kelly Criterion

The optimal fraction maximizes the expected logarithmic growth of capital (Kelly, 1956):

$$K = \frac{\text{edge}}{\text{odds} - 1} \quad \text{where} \quad \text{edge} = c \times \text{odds} - 1$$

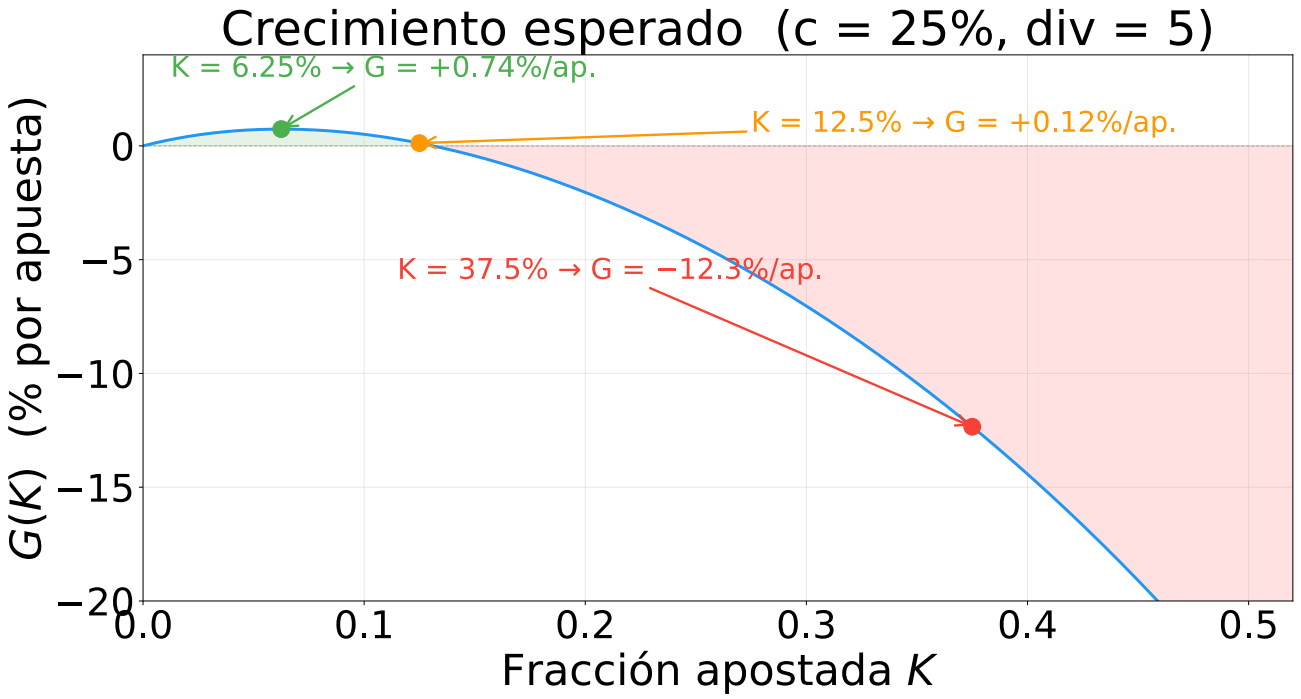
- $K$: fraction of capital to bet ($K < 0 \rightarrow$ don't bet)
- $c$: estimated probability of winning (our model)
- $\text{odds}$: gross payout per \$1 bet (stake included)
- $\text{edge} < 0$: unfavorable game; $= 0$: fair game; $> 0$: bet $K$

Warning (Benter, 1994): overestimating $c$ pushes $K$ upward — betting too much can cause negative capital growth.

# Horse racing: long-run consequences

Kelly maximizes the expected log growth rate per bet:

$$G(K) = c \cdot \ln(1 + (\text{odds} - 1) \cdot K) + (1 - c) \cdot \ln(1 - K)$$




| $K$ | $c$ est. | $G$/ bet | ×200 bets |
|---|---|---|---|
| 6.25% | 25% | +0.74% | ×4.4 |
| 12.5% | 30% | +0.12% | ×1.3 |
| 37.5% | 50% | −12.3% | ruin |

c real = 25%, odds = 5

# Matchmaking

# The matchmaking problem

We must decide which profiles to show each user *every day*.

It is an iterated problem:

- today's state depends on yesterday's state.
- it's not *that* important to solve it "one-shot"

# The matchmaking problem

We must decide which profiles to show each user *every day*.

It is an iterated problem:

- today's state depends on yesterday's state.
- it's not *that* important to solve it "one-shot"

Multiple divergent objectives:

- Maximize user engagement ($\Longrightarrow$ show aspirational candidates)
- Maximize matches formed ($\Longrightarrow$ show realistic candidates)

# The matchmaking problem

We must decide which profiles to show each user *every day*.

It is an iterated problem:

- today's state depends on yesterday's state.
- it's not *that* important to solve it "one-shot"

Multiple divergent objectives:

- Maximize user engagement ($\Longrightarrow$ show aspirational candidates)
- Maximize matches formed ($\Longrightarrow$ show realistic candidates)
- Maximize growth ($\Longrightarrow$ don't charge / "freemium" model)
- Maximize gross/net revenue ($\Longrightarrow$ paywall)

# The matchmaking problem

- We want to estimate $P(A \leftrightarrow B)$: probability of *mutual match*

# The matchmaking problem

- We want to estimate $P(A \leftrightarrow B)$: probability of *mutual match*
- We can estimate $P(A \rightarrow B)$: probability that A approves of B

# The matchmaking problem

- We want to estimate $P(A \leftrightarrow B)$: probability of *mutual match*
- We can estimate $P(A \rightarrow B)$: probability that A approves of B

Dilemma: probabilities are asymmetric.

Everyone wants Brad Pitt / Penélope Cruz ...

# The matchmaking problem

- We want to estimate $P(A \leftrightarrow B)$: probability of *mutual match*
- We can estimate $P(A \rightarrow B)$: probability that A approves of B

Dilemma: probabilities are asymmetric.

Everyone wants Brad Pitt / Penélope Cruz ... but they don't want everyone back.

# Directional vs. mutual probabilities

The true mutual probability has a correlation term: people who “offer” what others seek tend to “seek” similar things themselves.

$$P(A \leftrightarrow B) \neq P(A \rightarrow B) \times P(B \rightarrow A)$$

# Directional vs. mutual probabilities

The true mutual probability has a correlation term: people who "offer" what others seek tend to "seek" similar things themselves.

$$P(A \leftrightarrow B) \neq P(A \rightarrow B) \times P(B \rightarrow A)$$

But estimating $P(A \leftrightarrow B)$ is harder than just $P(A \rightarrow B)$, so we can assume

$$P(A \leftrightarrow B) \approx P(A \rightarrow B) \times P(B \rightarrow A)$$

# Directional vs. mutual probabilities

The true mutual probability has a correlation term: people who "offer" what others seek tend to "seek" similar things themselves.

$$P(A \leftrightarrow B) \neq P(A \rightarrow B) \times P(B \rightarrow A)$$

But estimating $P(A \leftrightarrow B)$ is harder than just $P(A \rightarrow B)$, so we can assume

$$P(A \leftrightarrow B) \approx P(A \rightarrow B) \times P(B \rightarrow A)$$

And estimate conditional probabilities with a single estimator:

$$P(A \leftrightarrow B \mid A \rightarrow B) = P(B \rightarrow A)$$

# Directional vs. mutual probabilities

When making *decisions* with the model, this interpretability is very useful...

# Directional vs. mutual probabilities

When making *decisions* with the model, this interpretability is very useful... even if it's wrong!.

*"All models are wrong, but some are useful." — Box (Box, 1976)*

# Gale-Shapley: Stable Matching

Classic algorithm (Gale & Shapley, 1962); Nobel Prize in Economics 2012 (Shapley & Roth)

1. Each A proposes to their top available candidate
2. Each B tentatively accepts the best proposer; rejects the rest
3. Rejected As propose to their next candidate
4. Repeat until everyone is paired

Guarantees stability: no rejected pair would both prefer to switch to each other.

# Gale-Shapley: Stable Matching

Classic algorithm (Gale & Shapley, 1962); Nobel Prize in Economics 2012 (Shapley & Roth)

1. Each A proposes to their top available candidate
2. Each B tentatively accepts the best proposer; rejects the rest
3. Rejected As propose to their next candidate
4. Repeat until everyone is paired

Guarantees stability: no rejected pair would both prefer to switch to each other.

Problems in dating:

- Requires "tentatively accepting" candidates — doesn't apply
- Optimizes stability, not global welfare
- The stable matching can be very unequal

  *The* proposing *side gets their best stable partner; the* receiving *side gets their worst.*

# Greedy Matching

1. Sort edges by $P(A \leftrightarrow B)$ descending
2. Add edge if both nodes are free
3. Guarantees in theory $\geq \frac{1}{2}$ of optimal (*1/2-approximation*)[1]

[1]

# Greedy Matching

1. Sort edges by $P(A \leftrightarrow B)$ descending
2. Add edge if both nodes are free
3. Guarantees in theory $\geq \frac{1}{2}$ of optimal (*1/2-approximation*)[1]

[1] but in practice it's usually much better.

# Greedy Matching

1. Sort edges by $P(A \leftrightarrow B)$ descending
2. Add edge if both nodes are free
3. Guarantees in theory $\geq \frac{1}{2}$ of optimal (*1/2-approximation*)[1]

Simple and intuitive, but can "spend" highly-demanded nodes on good-but-not-optimal pairs

[1] but in practice it's usually much better.

# Maximum Weight Matching

- Weighted complete graph: edge weight = $P(A \leftrightarrow B)$
- Find the matching that maximizes total weight

# Maximum Weight Matching

- Weighted complete graph: edge weight = $P(A \leftrightarrow B)$
- Find the matching that maximizes total weight

Advantage: maximizes global welfare

Counterintuitive insight: the globally optimal matching may NOT include the pair with the highest individual probability

# Maximum Weight Matching

- Weighted complete graph: edge weight = $P(A \leftrightarrow B)$
- Find the matching that maximizes total weight

Advantage: maximizes global welfare

Counterintuitive insight: the globally optimal matching may NOT include the pair with the highest individual probability

For bipartite graphs: Hungarian algorithm (Kuhn, 1955) — O($n^3$), exact.
For general graphs: Edmonds' blossom algorithm (slower, not needed here).

# Brute force with `scipy.minimize`

- Generic formulation: relaxed combinatorial optimization
- Flexible — the objective function can be changed easily
- Few iterations needed (50–100 usually suffice), but each iteration is expensive

# Brute force with `scipy.minimize`

- Generic formulation: relaxed combinatorial optimization
- Flexible — the objective function can be changed easily
- Few iterations needed (50–100 usually suffice), but each iteration is expensive

Trade-off: approaches the optimum but doesn't guarantee it, and is slower

# Brute force with `scipy.minimize`

- Generic formulation: relaxed combinatorial optimization
- Flexible — the objective function can be changed easily
- Few iterations needed (50–100 usually suffice), but **each iteration is expensive**

**Trade-off**: approaches the optimum but doesn't guarantee it, and is slower

Why not simply enumerate all possible matchings? For $n = 500$ users per group:

| Method | Operations | Feasible? |
|---|---|---|
| Hungarian | $500^3 \approx 1.25 \times 10^8$ | Yes — seconds |
| Exhaustive | $500! \approx 10^{\{1134\}}$ | No — the universe has $\approx 10^{\{80\}}$ atoms |

# Comparing approaches

| Algorithm | Optimizes | Optimal? | Practical? |
|---|---|---|---|
| Gale-Shapley | Stability | Yes (stab.) | Hard in dating |
| Greedy | Local welfare | $\geq \frac{1}{2}$ opt. | Yes, very fast |
| Max Weight | Global welfare | Yes | Yes |
| Brute force | Configurable | Approx. | Slow |

# Comparing approaches

| Algorithm | Optimizes | Optimal? | Practical? |
|---|---|---|---|
| Gale-Shapley | Stability | Yes (stab.) | Hard in dating |
| Greedy | Local welfare | $\geq \frac{1}{2}$ opt. | Yes, very fast |
| Max Weight | Global welfare | Yes | Yes |
| Brute force | Configurable | Approx. | Slow |

*Let's look at the code...*

# Results

# Synthetic data

- 1000 users, 6 dimensions (beauty, wealth, extraversion, intellect, adventure, romance)
- Features with variable norm — some users have "more of everything" ($r_{\text{norm}} = 0.92$ with received attractiveness)
- Normalized preferences (pure direction) with a universal attractiveness component
- AUC $\approx 0.74$ with Logistic Regression and Gradient Boosted Trees

# Main comparison (1000 users)

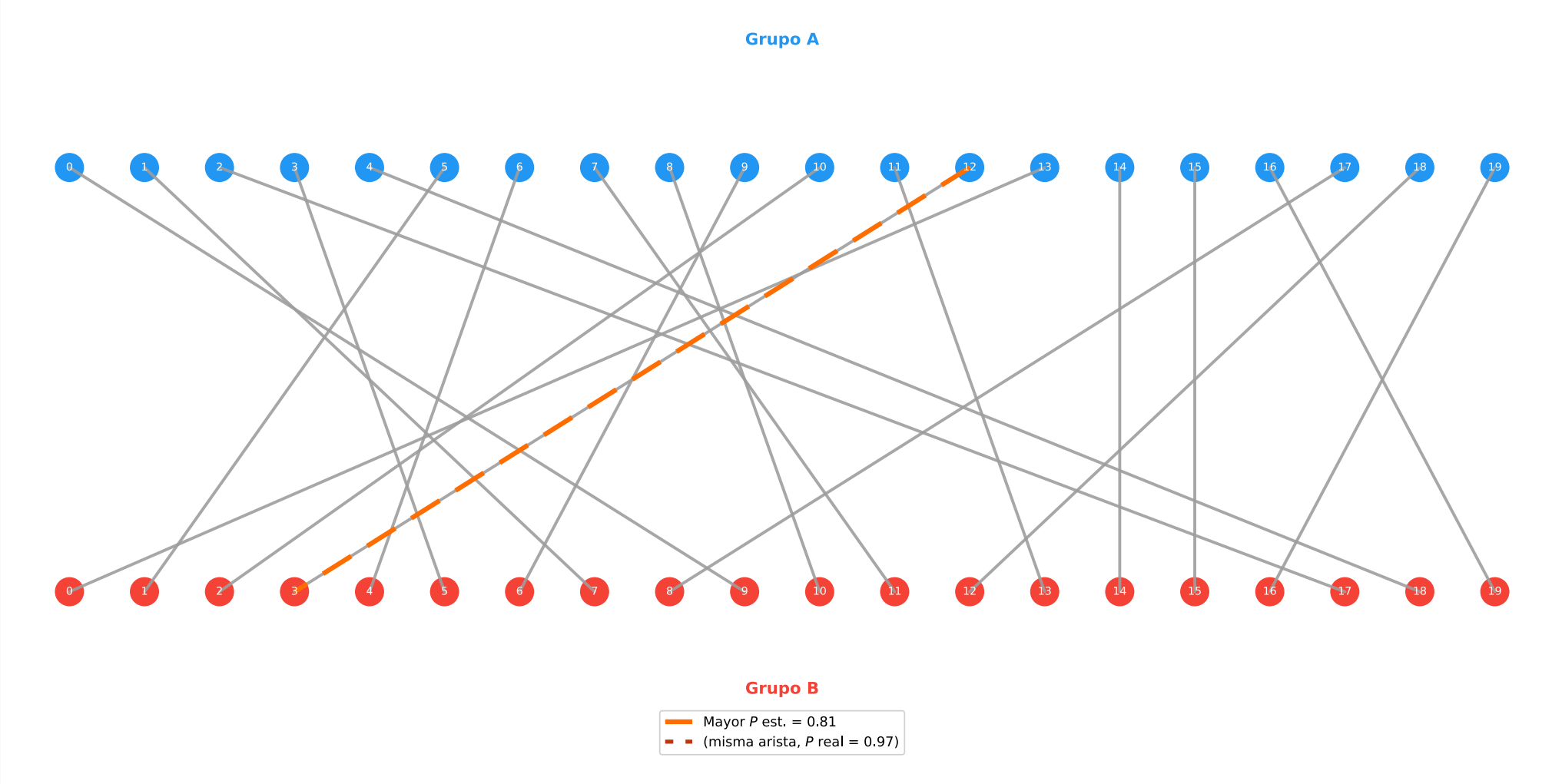
| Algorithm | Pairs | Weight | Time | % optimal |
|---|---|---|---|---|
| Gale-Shapley | 500 | 137.8 | 20ms | 91.2% |
| Greedy | 500 | 142.9 | 148ms | 94.6% |
| Max Weight | 500 | 151.1 | 107s | 100% |
| Scipy bipartite | 500 | 151.1 | 13ms | 100% |

# Full comparison (40 users)

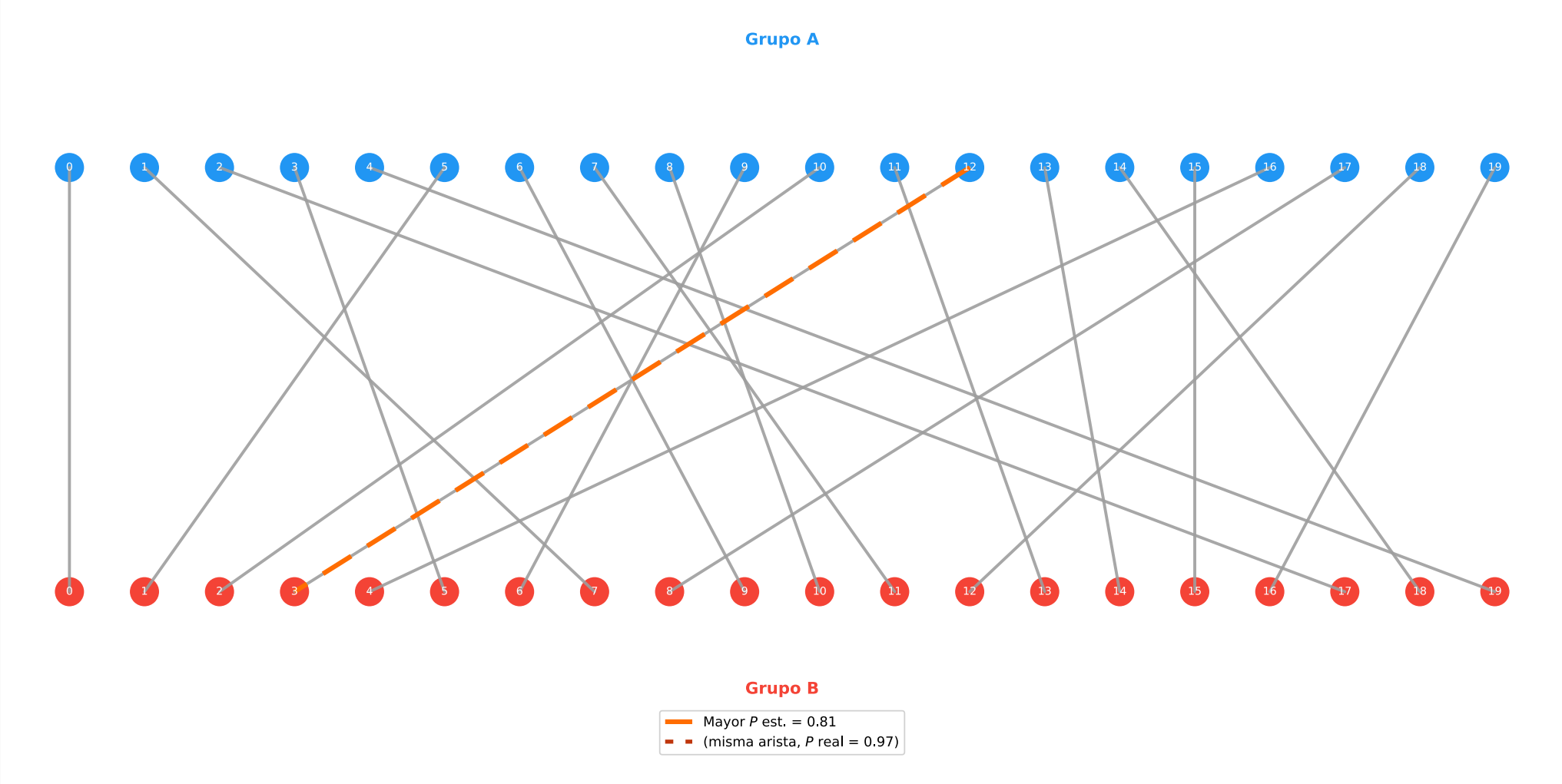
Max Weight, Scipy and Brute force agree (all optimal); GS loses 5%.

| Algorithm | Pairs | Weight | Time | % optimal |
|---|---|---|---|---|
| Gale-Shapley | 20 | 4.432 | < 0.1ms | 94.6% |
| Greedy | 20 | 4.485 | < 0.1ms | 95.7% |
| Max Weight | 20 | 4.687 | 6ms | 100% |
| Scipy bipartite | 20 | 4.687 | < 0.1ms | 100% |
| Brute force | 20 | 4.687 | 7.3s | 100% |

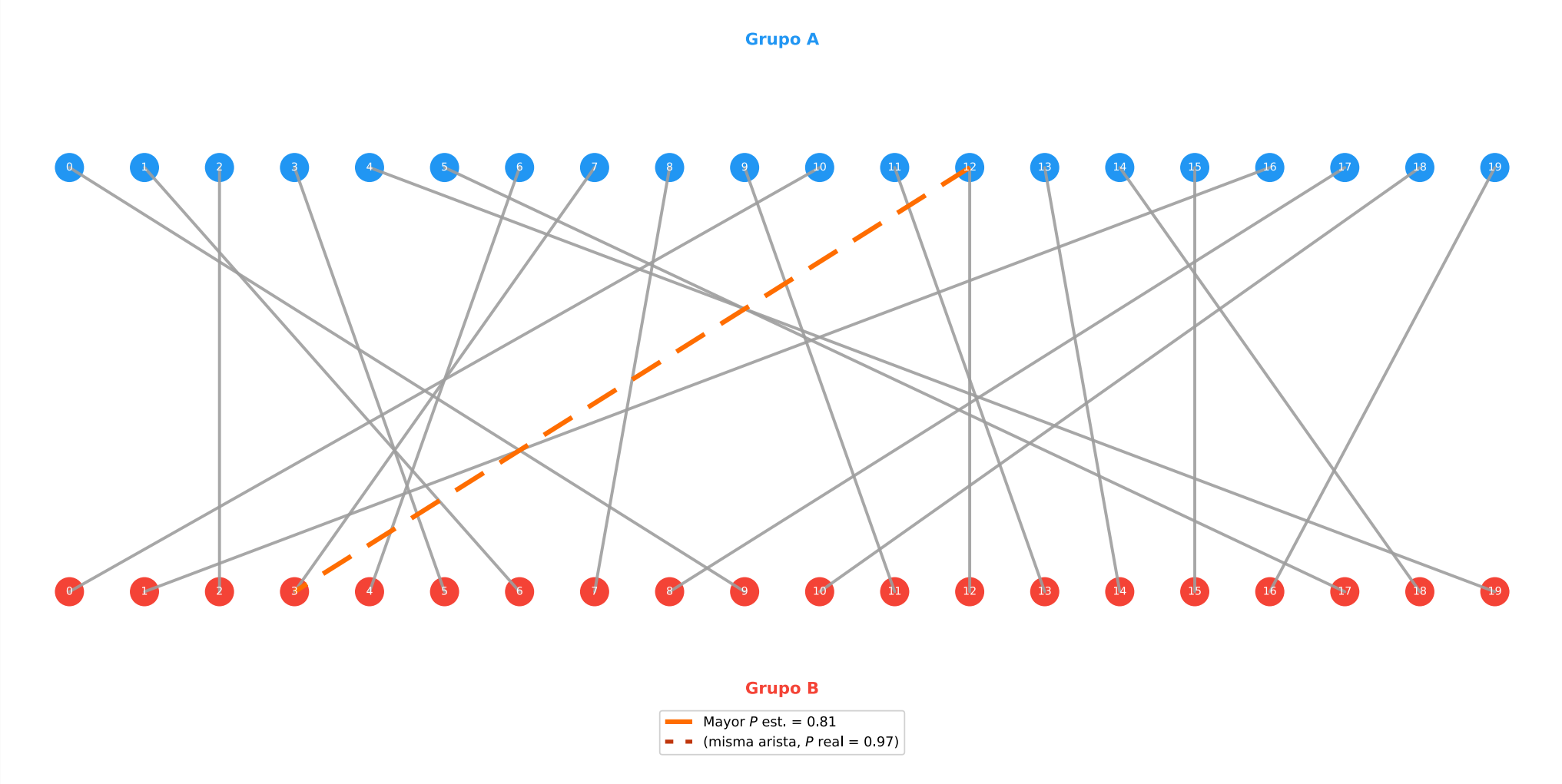
# Matchings in the graph — Gale-Shapley



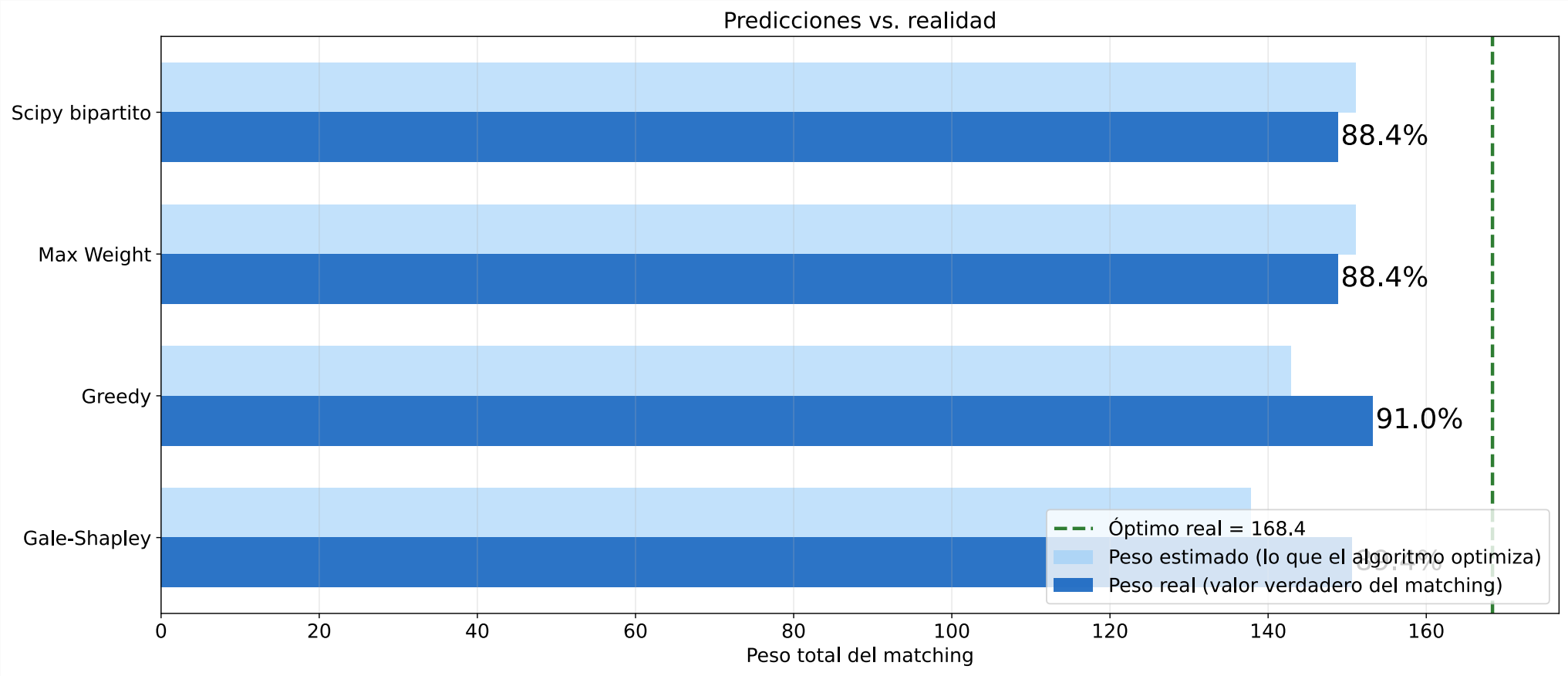
# Matchings in the graph — Greedy



# Matchings in the graph — Max Weight



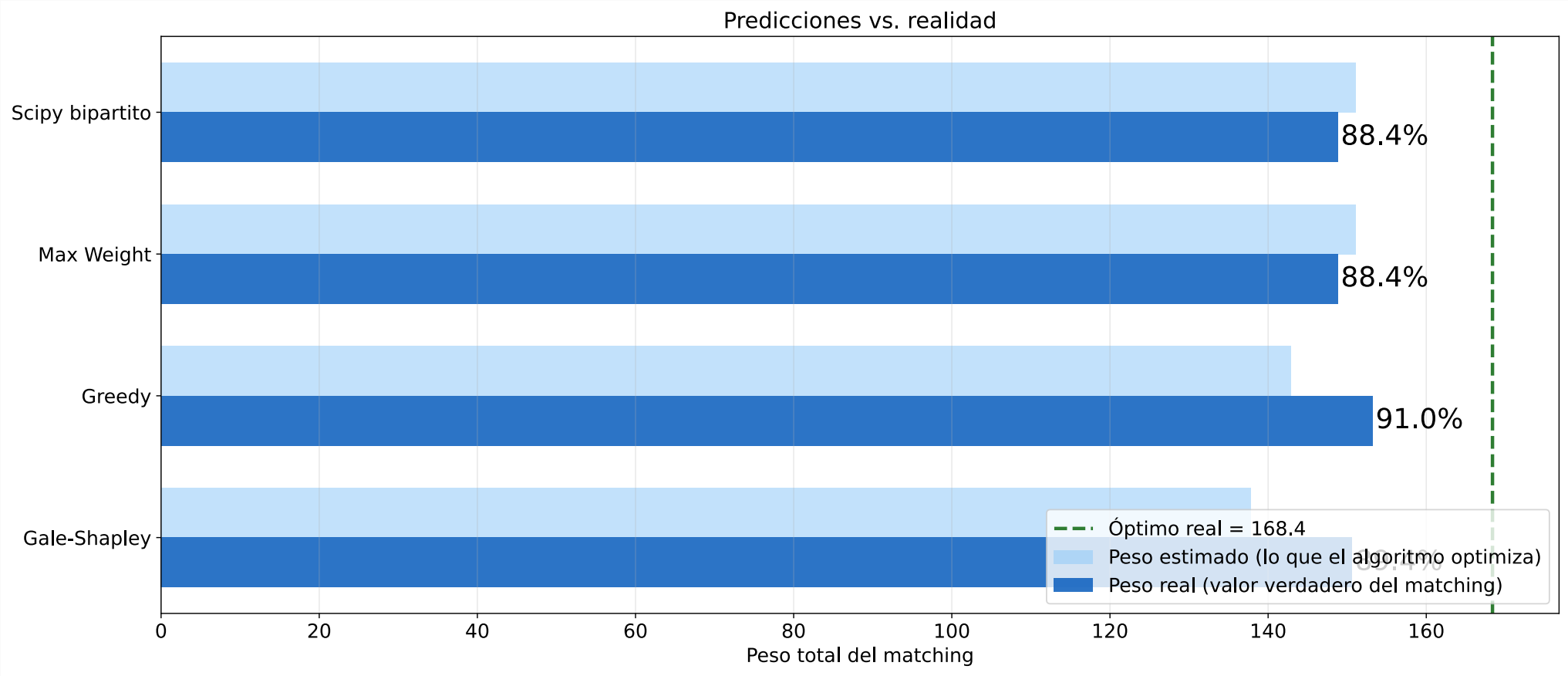
# Predictions vs. Reality




Real optimum = what we'd get with the true probabilities in hand (Scipy on $P_{\text{mut}}$).

# Predictions vs. Reality

# Predictions vs. Reality

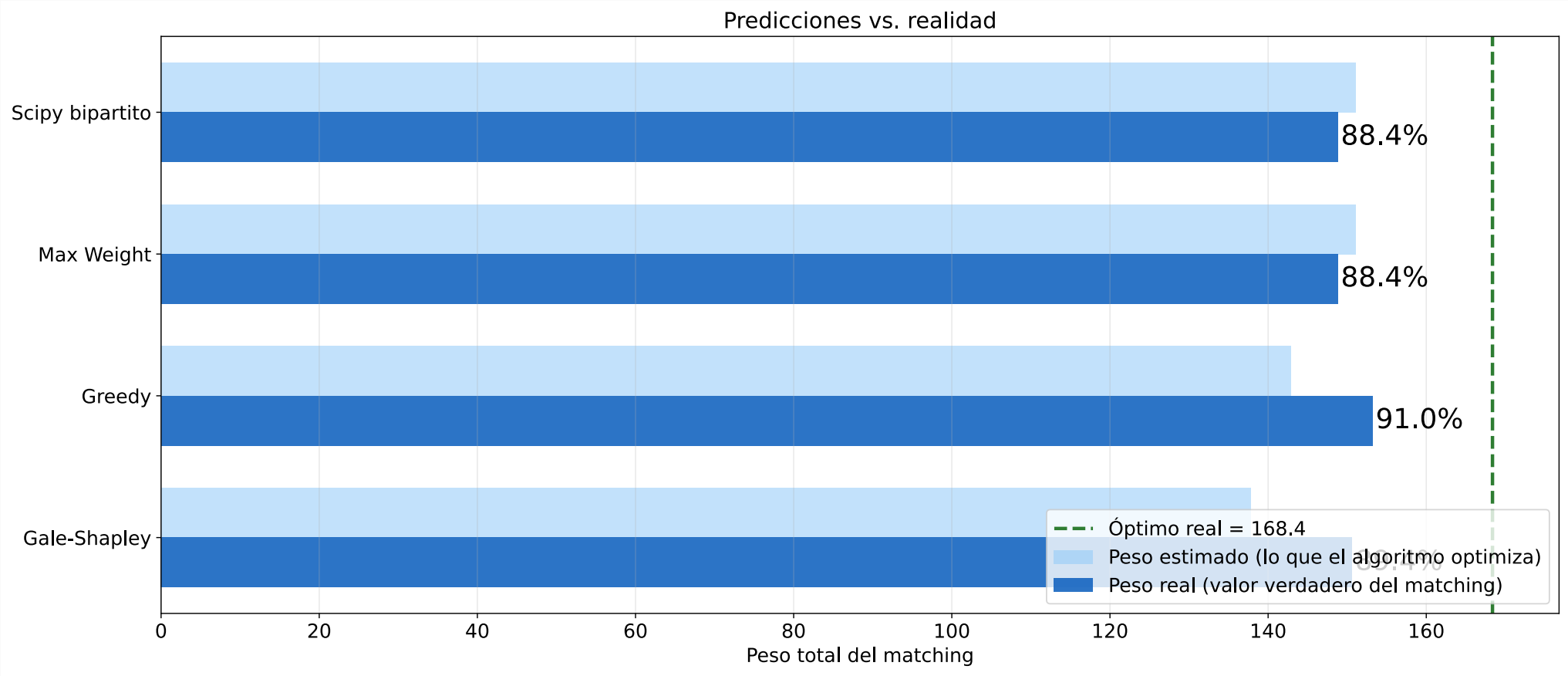



Real optimum = what we'd get with the true probabilities in hand (Scipy on $P_{\text{mut}}$).

# Predictions vs. Reality

| Algorithm | Est. weight | Real weight | % optimal |
|---|---|---|---|
| Gale-Shapley | 137.8 | 150.6 | 89.4% |
| Greedy | 142.9 | 153.2 | 91.0% |
| Max Weight | 151.1 | 148.8 | 88.4% |
| Scipy bipartite | 151.1 | 148.8 | 88.4% |

# Predictions vs. Reality




Real optimum = what we'd get with the true probabilities in hand (Scipy on $P_{\text{mut}}$).

# Predictions vs. Reality

| Algorithm | Est. weight | Real weight | % optimal |
|---|---|---|---|
| Gale-Shapley | 137.8 | 150.6 | 89.4% |
| Greedy | 142.9 | 153.2 | 91.0% |
| Max Weight | 151.1 | 148.8 | 88.4% |
| Scipy bipartite | 151.1 | 148.8 | 88.4% |

Paradox: Max Weight maximizes *estimated* weight but achieves the lowest *real* weight. Greedy, more conservative, outperforms all in effective terms.

# Key takeaways

1. The matching that maximizes global welfare may exclude the pair with the highest individual probability.
2. More “ambitious” algorithms are more fragile to estimation errors.

# Key takeaways

1. The matching that maximizes global welfare may exclude the pair with the highest individual probability.
2. More “ambitious” algorithms are more fragile to estimation errors.

This is the essence of decision-making in data science: optimize the whole system, not each component in isolation — but stay aware of the quality of your estimates.

# Thank you!

@capitantoto

github.com/capitantoto

gonzalo-barrera-borla

# References

# Bibliography


Benter, W. (1994). Computer Based Horse Race Handicapping and Wagering Systems: A Report. In D. B. Hausch, V. S. Y. Lo, & W. T. Ziemba (Eds.), *Efficiency of Racetrack Betting Markets: Efficiency of Racetrack Betting Markets* (pp. 183–198). Academic Press.

Box, G. E. P. (1976). Science and Statistics. *Journal of the American Statistical Association*, *71*(356), 791–799. https://doi.org/10.1080/01621459.1976.10480949

Gale, D., & Shapley, L. S. (1962). College Admissions and the Stability of Marriage. *The American Mathematical Monthly*, *69*(1), 9–15. https://doi.org/10.1080/00029890.1962.11989827

Kelly, J. L. (1956). A New Interpretation of Information Rate. *Bell System Technical Journal*, *35*(4), 917–926. https://doi.org/10.1002/j.1538-7305.1956.tb03809.x

Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

Pita, C. (2019). Optimal bidding: a dual approach. *Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1–6. http://papers.adkdd.org/2019/papers/adkdd19-pita-optimal.pdf